



Microservices Mastery with Node.js

Scalable Backend Architecture Essentials

2.

Agenda



1. Introduction and Fundamentals
2. Core Microservices Architecture
3. Reliability, Deployment, and Takeaways

3.



Introduction and Fundamentals

4.



Welcome to Microservices Mastery



From Monoliths to Microservices

Trace the shift from single-unit apps to distributed services.



Why Scalability Matters

Modern backends demand flexibility, resilience, and growth.



Course Roadmap

Journey from core concepts to production-ready systems.

5.



Microservices Fundamentals

Microservices Defined

Small, independent services focused on specific business capabilities.

Monolith vs SOA vs Microservices

Monolith: single codebase;
SOA: shared services;
Microservices: decentralized, loosely coupled.

Key Benefits & Challenges

Benefits: scalability, resilience, tech diversity.
Challenges: complexity, data consistency, deployment.

Where Microservices Excel

Ideal for large, evolving systems needing rapid scaling and independent deployments.

6.



Designing Service Boundaries



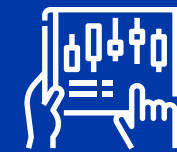
Decompose Monoliths with Domain-Driven Design

Break systems into bounded contexts for clear service boundaries.



Strangler Fig Pattern for Gradual Migration

Incrementally replace legacy components with new microservices.



Best Practices for Service Boundaries

Align services with business domains and minimize cross-service dependencies.

7.



Core Microservices Architecture

8.



Communication Patterns Unveiled



Synchronous vs Asynchronous Messaging

- Synchronous: Immediate response, tight coupling
- Asynchronous: Decoupled, scalable, uses queues



REST, gRPC, and Message Queues

- REST: Widely used HTTP-based API
- gRPC: High-performance, binary protocol
- Message Queues: Enable event-driven workflows



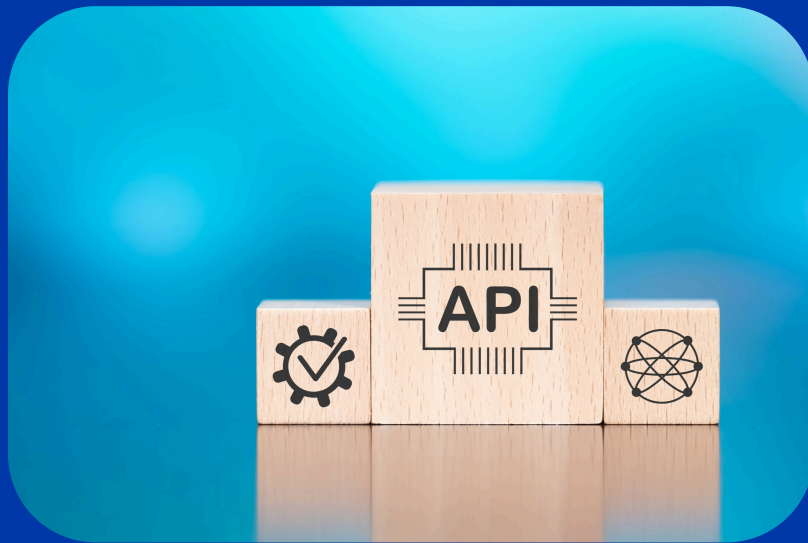
Request-Response vs Event-Based

- Request-response: Direct, predictable interactions
- Event-based: Reactive, supports distributed systems

9.



API Gateway and Service Discovery



API Gateway: Centralized Entry Point

Handles routing,
authentication, and request
aggregation



Service Discovery: Dynamic Connectivity

Enables services to locate
each other via registries



Scalability Through Decoupling

Gateways and discovery
reduce tight coupling and
enable scaling

10.



Data Management in Microservices

Database-per-Service Architecture

Each microservice owns its own database for autonomy.

Ensuring Data Consistency

Apply strategies like eventual consistency and replication.

Polyglot Persistence

Choose the best database type for each service's needs.

Avoid Shared Database Pitfalls

Prevent tight coupling and scaling issues by not sharing databases.

11.



Event-Driven and Saga Patterns

Event-Driven Architecture Fundamentals

Services communicate via events for loose coupling and scalability.

Role of Message Brokers

Message brokers like RabbitMQ and Kafka enable reliable event delivery.

Saga Pattern for Distributed Transactions

Sagas manage long-running, multi-service transactions with compensation.

Choreography vs Orchestration

Choreography: services react to events;
Orchestration: central coordinator manages flow.

12.



Reliability, Deployment, and Takeaways

13.



Resilience and Security Essentials

Reliability Through Resilience Patterns

- Use circuit breakers to prevent cascading failures
- Implement retries and bulkheads for fault isolation

Securing Microservices Communication

- Authenticate with JWT and OAuth2 for robust access control
- Enable mTLS for secure service-to-service connections

Protecting APIs and Managing Access

- Apply rate limiting to prevent abuse and overload
- Manage API keys for controlled external access

14.



Observability and Deployment



Comprehensive System Monitoring

- Track logs, metrics, and traces for real-time health insights.



Containerization for Scalability

- Package services with Docker for consistent deployment.
- Use Kubernetes to orchestrate and scale microservices.



Automated CI/CD Pipelines

- Streamline deployments and updates with automation.

15.



Key Takeaways and Next Steps

Unlock Scalable, Resilient Backends

Microservices architecture empowers growth and reliability.

Leverage a Modern Node.js Stack

Combine Node.js, TypeScript, and Express for robust services.

Accelerate Learning with Projects

Hands-on examples bridge theory and real-world application.

Build Your Own Distributed Systems

Apply proven patterns to create production-ready solutions.